

Workflow:

Backend-Setup

Datenbank · iCal-Sync · Stripe Connect · E-Mail

Komplette Implementierung in Replit

7

Setup-
Schritte

12

E-Mail-
Templates

4

API-
Routen

Backend-Setup für Klard auf Replit

Komplette Schritt-für-Schritt-Anleitung

Diese Anleitung führt Sie durch den vollständigen Aufbau des Klard-Backends auf Replit. Sie umfasst Datenbank-Migrationen, iCal-Kalender-Synchronisation, Stripe Connect für Auszahlungen, automatisierte E-Mail-Benachrichtigungen und die Webhook-Konfiguration. Alle Code-Beispiele sind direkt einsatzfähig — kopieren Sie sie 1:1 in Ihr Replit-Projekt.

Voraussetzungen

Was Sie brauchen	Wo Sie es bekommen
Replit-Konto (Pro empfohlen)	replit.com → Sign up
PostgreSQL-Datenbank	In Replit eingebaut oder Supabase (kostenlos bis 500 MB)
Stripe-Konto (Energy Impuls GmbH)	stripe.com/de → Konto registrieren (1-3 Tage Verifikation)
Domain klard.de	Bereits gesichert bei United Domains
E-Mail-Versand-Service	Resend (resend.com) oder Postmark (postmarkapp.com) — kostenlos bis 3.000/Monat

■ Zeitaufwand-Schätzung

Schritt 1–3 (Datenbank + Setup): ca. 2 Stunden
Schritt 4 (iCal-Sync): ca. 3 Stunden
Schritt 5 (Stripe Connect): ca. 4 Stunden (plus 1–3 Tage Stripe-Verifikation)
Schritt 6 (E-Mail-System): ca. 3 Stunden
Schritt 7 (Webhooks + Testing): ca. 2 Stunden
Gesamtaufwand: ca. 14 Stunden Entwicklungszeit (verteilt auf 3–5 Tage)

1

Replit-Projekt einrichten

Node.js-Projekt anlegen, Pakete installieren, Secrets hinterlegen

1.1 — Neues Replit-Projekt erstellen

Loggen Sie sich auf replit.com ein und klicken Sie auf **Create Repl**. Wählen Sie als Template **Node.js**. Nennen Sie das Projekt **klard-backend**.

1.2 — Benötigte npm-Pakete installieren

Öffnen Sie das Replit-Shell und führen Sie aus:

```
npm install express pg dotenv cors helmet
npm install stripe ical-generator node-ical
npm install resend handlebars
npm install nodemon --save-dev
```

Paket	Zweck
express	Web-Framework für API-Routen
pg	PostgreSQL-Client
stripe	Offizielle Stripe-SDK für Connect, Checkout, Webhooks
ical-generator	Erzeugt iCal-Feeds (.ics) für Anbieter-Kalender
node-ical	Parst externe iCal-URLs für Sperr-Slot-Import
resend	E-Mail-Versand-Service (3.000/Monat kostenlos)
handlebars	Template-Engine für E-Mails und Rechnungen

1.3 — Secrets in Replit anlegen

Öffnen Sie in Replit links die **Secrets**-Sektion. Diese Werte gehören NICHT in den Code, sondern hier hin:

Secret-Key	Beispiel-Wert	Wo Sie ihn finden
DATABASE_URL	postgres://user:pass@host:5432/klard	Replit DB oder Supabase Settings
STRIPE_SECRET_KEY	sk_test_51Abc... (Test) / sk_live_... (Live)	Stripe Dashboard → Developers → API keys
STRIPE_PUBLISHABLE_KEY	pk_test_51Abc...	Stripe Dashboard → Developers → API keys
STRIPE_WEBHOOK_SECRET	whsec_xyz...	Wird in Schritt 5 angelegt
RESEND_API_KEY	re_abc123...	resend.com/api-keys
FROM_EMAIL	hello@klard.de	Ihre verifizierte Domain bei Resend
BASE_URL	https://klard.de	Ihre Production-Domain

⚠ Sicherheit

Die Secret-Werte NIEMALS direkt in den Code schreiben oder ins Git-Repository committen. Replit Secrets ist genau dafür da. Greifen Sie im Code auf `process.env.STRIPE_SECRET_KEY` zu.

2

Datenbank-Schema anlegen

Tabellen für Anbieter, Buchungen, iCal und Stripe

2.1 — Basis-Schema (aus klard-seed-v2.sql)

Importieren Sie zuerst das v2-Seed-Skript mit Branchen, Kategorien, Service-Templates (siehe Datei klard-seed-v2.sql). Es enthält bereits 8 Branchen mit 153 Leistungen.

2.2 — Erweiterungs-Migrationen für iCal + Stripe + E-Mail

Führen Sie diese SQL-Befehle in Replit DB oder Supabase aus:

```
-- iCal-Sync Felder
ALTER TABLE providers ADD COLUMN IF NOT EXISTS ical_token TEXT UNIQUE;
ALTER TABLE providers ADD COLUMN IF NOT EXISTS external_ical_url TEXT;

-- Stripe Connect Felder
ALTER TABLE providers ADD COLUMN IF NOT EXISTS stripe_account_id TEXT UNIQUE;
ALTER TABLE providers ADD COLUMN IF NOT EXISTS stripe_onboarded_at TIMESTAMPTZ;
ALTER TABLE providers ADD COLUMN IF NOT EXISTS commission_rate DECIMAL(4,3) DEFAULT 0.07;

-- Token für iCal-Feed automatisch generieren
UPDATE providers SET ical_token = encode(gen_random_bytes(24), 'hex')
WHERE ical_token IS NULL;

-- Externe Sperr-Slots
CREATE TABLE IF NOT EXISTS blocked_slots (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  provider_id UUID NOT NULL REFERENCES providers(id) ON DELETE CASCADE,
  starts_at   TIMESTAMPTZ NOT NULL,
  ends_at     TIMESTAMPTZ NOT NULL,
  source      TEXT NOT NULL CHECK (source IN ('external_ical', 'manual')),
  external_id TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_blocked_provider_time
ON blocked_slots(provider_id, starts_at, ends_at);
```

```
-- Kunden-Tabelle
CREATE TABLE IF NOT EXISTS customers (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL UNIQUE,
  name        TEXT,
  phone       TEXT,
  company_name TEXT,
  street      TEXT,
  zip         TEXT,
  city        TEXT,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Buchungs-Tabelle
CREATE TABLE IF NOT EXISTS bookings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  provider_id  UUID NOT NULL REFERENCES providers(id),
  customer_id  UUID REFERENCES customers(id),
  service_id   UUID REFERENCES provider_services(id),
  customer_name TEXT,
  customer_email TEXT NOT NULL,
  starts_at    TIMESTAMPTZ NOT NULL,
  ends_at      TIMESTAMPTZ NOT NULL,
  price_cents  INTEGER NOT NULL,
  status       TEXT NOT NULL DEFAULT 'pending_payment'
              CHECK (status IN ('pending_payment', 'confirmed',
                                'cancelled', 'completed', 'no_show')),
  stripe_payment_intent_id TEXT UNIQUE,
  stripe_session_id        TEXT UNIQUE,
  confirmed_at             TIMESTAMPTZ,
  cancelled_at             TIMESTAMPTZ,
  created_at               TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_bookings_provider ON bookings(provider_id);
CREATE INDEX IF NOT EXISTS idx_bookings_status ON bookings(status);
```

```
-- E-Mail-Log (Tracking, was wann verschickt wurde)
CREATE TABLE IF NOT EXISTS email_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  template_id TEXT NOT NULL,
  recipient   TEXT NOT NULL,
  related_id  UUID,
  status      TEXT NOT NULL DEFAULT 'sent',
  resend_id   TEXT,
  sent_at     TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_email_log_recipient ON email_log(recipient);
```

✓ Schema-Verifizierung

Nach den Migrationen prüfen Sie mit: `SELECT COUNT(*) FROM branches;` (sollte 8 zurückgeben).
 Und `SELECT COUNT(*) FROM service_templates;` (sollte 153 zurückgeben).

3

Backend-Grundgerüst

Express-Server mit Datenbank-Verbindung

3.1 — Datei: index.js (Hauptserver)

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const { Pool } = require('pg');

const app = express();
const db = new Pool({ connectionString: process.env.DATABASE_URL });

// Wichtig: Webhook-Route VOR express.json() registrieren (raw body needed)
const stripeWebhookRouter = require('./routes/stripe-webhook');
app.use('/stripe/webhook', stripeWebhookRouter);

// Normaler Middleware
app.use(helmet());
app.use(cors({ origin: process.env.BASE_URL }));
app.use(express.json());

// Routen-Module
app.use('/api/calendar', require('./routes/calendar'));
app.use('/api/stripe', require('./routes/stripe'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/providers', require('./routes/providers'));

// Health-Check
app.get('/health', (req, res) => res.json({ ok: true }));

// Globaler Error-Handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Internal Server Error' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Klard-Backend läuft auf Port ${PORT}`));

module.exports = { db };
```

3.2 — Datei: db.js (PostgreSQL-Helfer)

```

const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

module.exports = {
  query: (text, params) => pool.query(text, params),
  one: async (text, params) => {
    const { rows } = await pool.query(text, params);
    if (rows.length === 0) throw new Error('No rows returned');
    return rows[0];
  },
  oneOrNull: async (text, params) => {
    const { rows } = await pool.query(text, params);
    return rows[0] || null;
  },
  any: async (text, params) => {
    const { rows } = await pool.query(text, params);
    return rows;
  },
  none: async (text, params) => {
    await pool.query(text, params);
  },
};

```

3.3 — Projekt-Struktur

```

klard-backend/
├── index.js           # Express-Hauptserver
├── db.js             # Datenbank-Helfer
├── package.json
├── routes/
│   ├── calendar.js   # iCal-Feed + externe Sync
│   ├── stripe.js     # Stripe Connect Onboarding
│   ├── stripe-webhook.js # Stripe Webhook-Empfang
│   ├── bookings.js   # Buchungs-Logik
│   └── providers.js   # Anbieter-CRUD
├── services/
│   ├── email.js      # Resend-Wrapper
│   └── email-templates/ # Handlebars-Templates
│       ├── booking-confirmation.hbs
│       ├── booking-reminder.hbs
│       └── ... (12 Templates)
└── public/
    └── klard-plattform.html # Frontend (von Klard bereitgestellt)

```

4

iCal-Feed-Integration

Kalender-Sync in beide Richtungen

4.1 — Konzept

iCal-Sync läuft in zwei Richtungen:

Export (Klard → Anbieter-Kalender): Klard stellt eine eindeutige Feed-URL pro Anbieter bereit. Der Anbieter abonniert sie in Google/Outlook/Apple Kalender. Neue Klard-Buchungen erscheinen automatisch im persönlichen Kalender.

Import (Anbieter-Kalender → Klard): Anbieter trägt seine externe iCal-URL ein. Klard fragt sie alle 15 Minuten ab und blockiert belegte Zeiten für Klard-Buchungen. So bleiben private Termine privat — aber Buchungs-Kollisionen werden verhindert.

4.2 — Datei: routes/calendar.js

```

const express = require('express');
const ical = require('ical-generator').default;
const nodeIcal = require('node-ical');
const crypto = require('crypto');
const db = require('../db');

const router = express.Router();

// =====
// EXPORT: GET /calendar/feed/:token.ics
// =====
router.get('/feed/:token.ics', async (req, res) => {
  const { token } = req.params;
  const provider = await db.oneOrNull(
    `SELECT id, company_name FROM providers WHERE ical_token = $1`,
    [token]
  );
  if (!provider) return res.status(404).send('Feed not found');

  const bookings = await db.any(`
    SELECT b.id, b.starts_at, b.ends_at, b.customer_name,
           st.name AS service_name
    FROM bookings b
    LEFT JOIN provider_services ps ON ps.id = b.service_id
    LEFT JOIN service_templates st ON st.id = ps.service_template_id
    WHERE b.provider_id = $1
           AND b.status = 'confirmed'
           AND b.starts_at > NOW() - INTERVAL '30 days'
  `, [provider.id]);

  const cal = ical({
    name: `Klard - ${provider.company_name}`,
    prodId: { company: 'Klard', product: 'Booking', language: 'DE' },
    timezone: 'Europe/Berlin',
    ttl: 60 * 15,
  });

  bookings.forEach(b => {
    cal.createEvent({
      id: `klard-booking-${b.id}@klard.de`,
      start: b.starts_at,
      end: b.ends_at,
      summary: `${b.service_name} - ${b.customer_name}`,
      description: `Klard-Buchung\nKunde: ${b.customer_name}`,
      url: `${process.env.BASE_URL}/booking/${b.id}`,
    });
  });

  res.setHeader('Content-Type', 'text/calendar; charset=utf-8');
  res.send(cal.toString());
});

// Token neu generieren
router.post('/regenerate-token', requireAuth, async (req, res) => {
  const newToken = crypto.randomBytes(24).toString('hex');
  await db.none('UPDATE providers SET ical_token = $1 WHERE id = $2',
    [newToken, req.user.providerId]);
  res.json({ token: newToken });
});

module.exports = router;

```

4.3 — Externer Kalender abfragen (Datei: services/ical-sync.js)

```

const nodeIcal = require('node-ical');
const db = require('../db');

async function syncExternalCalendars() {
  const providers = await db.any(
    'SELECT id, external_ical_url FROM providers WHERE external_ical_url IS NOT NULL'
  );

  for (const p of providers) {
    try {
      const events = await nodeIcal.async.fromURL(p.external_ical_url);

      // Alte externe Sperrungen löschen
      await db.none(
        `DELETE FROM blocked_slots
        WHERE provider_id = $1 AND source = 'external_ical'`,
        [p.id]
      );

      for (const key in events) {
        const e = events[key];
        if (e.type !== 'VEVENT' || !e.start || !e.end) continue;
        if (new Date(e.end) < new Date()) continue; // nur zukünftige

        await db.none(
          `INSERT INTO blocked_slots
          (provider_id, starts_at, ends_at, source, external_id)
          VALUES ($1, $2, $3, 'external_ical', $4)
          `, [p.id, e.start, e.end, e.uid || key]);
      }
      console.log(`✓ iCal sync ok für Anbieter ${p.id}`);
    } catch (err) {
      console.error(`✗ iCal sync fehlgeschlagen ${p.id}:`, err.message);
    }
  }
}

// Initial + alle 15 Minuten
syncExternalCalendars();
setInterval(syncExternalCalendars, 15 * 60 * 1000);

module.exports = { syncExternalCalendars };

```

■ Cron-Jobs auf Replit

Damit `setInterval` auch beim Schlafen des Repls läuft, aktivieren Sie die Replit Always-On-Funktion (Pro-Plan) oder nutzen einen externen Cron-Service wie cron-job.org.
 Alternative: ein zweiter Replit-Worker, der via `curl` alle 15 Min. einen internen Endpoint `/api/calendar/sync-now` aufruft.

5

Stripe Connect Setup

Auszahlungen direkt an Anbieter, Klard erhält Provision

5.1 — Im Stripe-Dashboard vorbereiten

Loggen Sie sich auf dashboard.stripe.com ein. In der linken Sidebar:

1. Connect → Erste Schritte
2. Wählen: "Plattform oder Marktplatz"
3. Branchen-Kategorie: "Professional Services"
4. Modell: Anbieter sind Verkäufer, Plattform behält Provision
5. Sub-Account-Typ: "Express" wählen (einfachstes Onboarding)
6. Connect-Onboarding ausfüllen (ca. 30 Min.)
7. Developers → API keys → Test-Mode-Keys kopieren

5.2 — Datei: routes/stripe.js

```

const express = require('express');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const db = require('../db');
const router = express.Router();

// =====
// Onboarding: Express-Account erstellen, Onboarding-Link generieren
// =====
router.post('/onboard', requireAuth, async (req, res) => {
  const providerId = req.user.providerId;
  let provider = await db.one('SELECT * FROM providers WHERE id = $1',
    [providerId]);

  // Vorhandenen Account verwenden oder neuen erstellen
  let stripeAccountId = provider.stripe_account_id;

  if (!stripeAccountId) {
    const account = await stripe.accounts.create({
      type: 'express',
      country: 'DE',
      email: provider.email,
      business_type: provider.legal_form === 'gmbh' ? 'company' : 'individual',
      capabilities: {
        card_payments: { requested: true },
        transfers: { requested: true },
      },
      business_profile: {
        name: provider.company_name,
        url: `${process.env.BASE_URL}/anbieter/${provider.slug}`,
        mcc: '7392', // "Management, Consulting, Public Relations"
      },
    });
    stripeAccountId = account.id;
    await db.none(
      'UPDATE providers SET stripe_account_id = $1 WHERE id = $2',
      [stripeAccountId, providerId]
    );
  }

  const accountLink = await stripe.accountLinks.create({
    account: stripeAccountId,
    refresh_url: `${process.env.BASE_URL}/dashboard/payouts?refresh=1`,
    return_url: `${process.env.BASE_URL}/dashboard/payouts?done=1`,
    type: 'account_onboarding',
  });

  res.json({ url: accountLink.url });
});

// Status prüfen
router.get('/status', requireAuth, async (req, res) => {
  const p = await db.one('SELECT stripe_account_id FROM providers WHERE id = $1',
    [req.user.providerId]);
  if (!p.stripe_account_id) return res.json({ status: 'not_started' });

  const acc = await stripe.accounts.retrieve(p.stripe_account_id);
  res.json({
    status: acc.details_submitted ? 'active' : 'incomplete',
    can_receive_payouts: acc.payouts_enabled,
    can_accept_payments: acc.charges_enabled,
    requirements: acc.requirements,
  });
});

module.exports = router;

```

5.3 — Buchung mit Zahlung verarbeiten (routes/bookings.js)

```

router.post('/checkout', async (req, res) => {
  const { providerId, serviceId, customerEmail, startsAt, endsAt } = req.body;

  const data = await db.one(`
    SELECT p.stripe_account_id, p.company_name, p.commission_rate,
           ps.price, st.name AS service_name
    FROM providers p
    JOIN provider_services ps ON ps.provider_id = p.id
    JOIN service_templates st ON st.id = ps.service_template_id
    WHERE p.id = $1 AND ps.id = $2
  `, [providerId, serviceId]);

  if (!data.stripe_account_id) {
    return res.status(400).json({ error: 'Anbieter hat Zahlungen nicht aktiviert.' });
  }

  const priceCents = Math.round(data.price * 100);
  const feeCents = Math.round(priceCents * data.commission_rate);

  const booking = await db.one(`
    INSERT INTO bookings
      (provider_id, customer_email, service_id, starts_at, ends_at,
       price_cents, status)
    VALUES ($1, $2, $3, $4, $5, $6, 'pending_payment')
    RETURNING id
  `, [providerId, customerEmail, serviceId, startsAt, endsAt, priceCents]);

  const session = await stripe.checkout.sessions.create({
    payment_method_types: ['card', 'sepa_debit', 'sofort'],
    mode: 'payment',
    customer_email: customerEmail,
    line_items: [{
      price_data: {
        currency: 'eur',
        product_data: { name: data.service_name, description: data.company_name },
        unit_amount: priceCents,
      },
      quantity: 1,
    }],
    payment_intent_data: {
      application_fee_amount: feeCents,
      transfer_data: { destination: data.stripe_account_id },
      metadata: { booking_id: booking.id, klard_provider: data.company_name },
    },
    success_url: `${process.env.BASE_URL}/booking/success?id=${booking.id}`,
    cancel_url: `${process.env.BASE_URL}/booking/cancel?id=${booking.id}`,
  });

  await db.none('UPDATE bookings SET stripe_session_id = $1 WHERE id = $2',
    [session.id, booking.id]);

  res.json({ checkoutUrl: session.url });
});

```

5.4 — Webhook: routes/stripe-webhook.js

```

const express = require('express');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const db = require('../db');
const { sendEmail } = require('../services/email');

const router = express.Router();

// Wichtig: raw body (kein JSON-Parse), für Signatur-Verifizierung
router.post('/', express.raw({ type: 'application/json' }), async (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;
  try {
    event = stripe.webhooks.constructEvent(
      req.body, sig, process.env.STRIPE_WEBHOOK_SECRET
    );
  } catch (err) {
    console.error('Webhook verify failed:', err.message);
    return res.status(400).send(`Webhook Error: ${err.message}`);
  }

  switch (event.type) {
    case 'checkout.session.completed': {
      const session = event.data.object;
      const pi = await stripe.paymentIntents.retrieve(session.payment_intent);
      const bookingId = pi.metadata.booking_id;

      await db.none(`
        UPDATE bookings
        SET status = 'confirmed', stripe_payment_intent_id = $1, confirmed_at = NOW()
        WHERE id = $2
      `, [session.payment_intent, bookingId]);

      // E-Mail an Kunde + Anbieter
      const b = await db.one(`
        SELECT b.*, p.email AS provider_email, p.company_name
        FROM bookings b JOIN providers p ON p.id = b.provider_id
        WHERE b.id = $1`, [bookingId]);
      await sendEmail('booking_confirmation_customer', b.customer_email, b);
      await sendEmail('booking_confirmation_provider', b.provider_email, b);
      break;
    }
    case 'account.updated': {
      const acc = event.data.object;
      if (acc.details_submitted && acc.payouts_enabled) {
        await db.none(`
          UPDATE providers SET stripe_onboarded_at = NOW()
          WHERE stripe_account_id = $1 AND stripe_onboarded_at IS NULL
        `, [acc.id]);
        const p = await db.oneOrNull('SELECT email FROM providers WHERE stripe_account_id = $1', [acc.id]);
        if (p) await sendEmail('stripe_activated', p.email, p);
      }
      break;
    }
    case 'payment_intent.payment_failed': {
      const pi = event.data.object;
      const bookingId = pi.metadata?.booking_id;
      if (bookingId) {
        await db.none('UPDATE bookings SET status = 'cancelled', cancelled_at = NOW() WHERE id = $1', [bookingId]);
      }
      break;
    }
  }

  res.json({ received: true });
});

module.exports = router;

```

5.5 — Webhook im Stripe-Dashboard registrieren

1. Stripe Dashboard → Developers → Webhooks → Add endpoint
2. URL: `https://klard.de/stripe/webhook`
(oder Replit-Test-URL für Testing: `https://klard-backend.ihr-name.repl.co/stripe/webhook`)
3. Events auswählen:
 - `checkout.session.completed`
 - `payment_intent.payment_failed`
 - `account.updated`
 - `charge.refunded`
4. "Add endpoint" → kopieren Sie das "Signing secret" (`whsec_...`)
5. In Replit Secrets als `STRIPE_WEBHOOK_SECRET` hinterlegen

■ Test mit Stripe Test-Modus

Test-Kreditkarte: 4242 4242 4242 4242, beliebiges Datum, beliebige CVC.

Test-IBAN für SEPA: DE89370400440532013000.

Alle Test-Zahlungen erscheinen sofort im Dashboard ohne echtes Geld zu bewegen.

6

E-Mail-Benachrichtigungen

Resend einrichten, 12 Templates für alle Use Cases

6.1 — Resend-Account & Domain-Verifizierung

Resend ist ein moderner Transaktions-E-Mail-Dienst (von den Vercel/Next.js-Erfindern), 3.000 E-Mails/Monat kostenlos, einfache API. Alternativ: Postmark, SendGrid, Mailgun.

1. resend.com → Sign up mit E-Mail
2. API Keys → Create API Key → in Replit als RESEND_API_KEY speichern
3. Domains → Add Domain → klard.de
4. DNS-Einträge bei United Domains hinterlegen:
 - TXT @ "v=spf1 include:_spf.resend.com ~all"
 - TXT resend._domainkey "<DKIM-Key von Resend>"
 - MX @ "10 feedback-smtp.eu-west-1.amazonses.com"
5. Bei Resend → "Verify" klicken (DNS-Propagation 1-24 Std.)
6. From-E-Mail: hello@klard.de freischalten

6.2 — Datei: services/email.js (Versand-Wrapper)

```

const { Resend } = require('resend');
const fs = require('fs');
const path = require('path');
const Handlebars = require('handlebars');
const db = require('../db');

const resend = new Resend(process.env.RESEND_API_KEY);

// Templates laden (Cache)
const TEMPLATES = {};
const TEMPLATE_DIR = path.join(__dirname, 'email-templates');
fs.readdirSync(TEMPLATE_DIR).forEach(file => {
  if (file.endsWith('.hbs')) {
    const id = file.replace('.hbs', '');
    const html = fs.readFileSync(path.join(TEMPLATE_DIR, file), 'utf8');
    TEMPLATES[id] = Handlebars.compile(html);
  }
});

// Mapping: Template-ID → Betreff
const SUBJECTS = {
  welcome_provider: 'Willkommen bei Klard, {{providerName}}!',
  welcome_customer: 'Willkommen bei Klard, {{customerName}}',
  profile_activated: 'Ihr Klard-Profil ist live',
  booking_confirmation_customer: 'Buchung bestätigt: {{serviceName}} am {{bookingDate}}',
  booking_confirmation_provider: 'Neue Buchung: {{customerName}} am {{bookingDate}}',
  booking_reminder_24h: 'Erinnerung: Ihr Termin morgen mit {{providerName}}',
  booking_reminder_1h: 'Ihr Termin beginnt in 1 Stunde',
  booking_cancelled_by_customer: 'Buchung storniert: {{serviceName}}',
  booking_cancelled_by_provider: 'Termin abgesagt - Erstattung folgt',
  invoice_ready: 'Ihre Klard-Rechnung Nr. {{invoiceNumber}}',
  stripe_activated: 'Auszahlungen sind jetzt aktiv',
  payment_failed: 'Zahlungsproblem bei Ihrer Buchung',
};

async function sendEmail(templateId, recipient, data) {
  const template = TEMPLATES[templateId];
  if (!template) throw new Error(`Template '${templateId}' nicht gefunden`);

  const subjectTpl = Handlebars.compile(SUBJECTS[templateId] || 'Klard');
  const subject = subjectTpl(data);
  const html = template(data);

  try {
    const result = await resend.emails.send({
      from: `Klard <${process.env.FROM_EMAIL}>`,
      to: recipient,
      subject,
      html,
    });
  }

  await db.none(`
    INSERT INTO email_log (template_id, recipient, related_id, status, resend_id)
    VALUES ($1, $2, $3, 'sent', $4)
  `, [templateId, recipient, data.id || data.bookingId || null, result.data?.id]);

  return result;
} catch (err) {
  await db.none(`
    INSERT INTO email_log (template_id, recipient, status)
    VALUES ($1, $2, 'failed')
  `, [templateId, recipient]);
  console.error('E-Mail-Versand fehlgeschlagen:', err);
  throw err;
}
}

module.exports = { sendEmail };

```

6.3 — Die 12 E-Mail-Templates im Überblick

Template-ID	Empfänger	Auslöser
welcome_provider	Anbieter	Nach Onboarding-Abschluss
welcome_customer	Kunde	Nach erster Buchung
profile_activated	Anbieter	Nach Admin-Freischaltung (24h)
booking_confirmation_customer	Kunde	Nach erfolgreicher Zahlung
booking_confirmation_provider	Anbieter	Nach erfolgreicher Zahlung
booking_reminder_24h	Kunde	24h vor Termin (Cron)
booking_reminder_1h	Kunde + Anbieter	1h vor Termin (Cron)
booking_cancelled_by_customer	Kunde + Anbieter	Kunden-Stornierung
booking_cancelled_by_provider	Kunde	Anbieter sagt ab → Erstattung
invoice_ready	Kunde	Nach erbrachter Leistung (status=completed)
stripe_activated	Anbieter	Stripe-Onboarding abgeschlossen
payment_failed	Kunde	Zahlung fehlgeschlagen

■ Die kompletten HTML-Templates

Alle 12 E-Mail-Templates inkl. fertiges HTML, Handlebars-Platzhalter und Versand-Logik liefere ich separat in der Datei `klard-email-templates.zip`. Sie kopieren den Ordner `services/email-templates/` einfach in Ihr Replit-Projekt.

7

Cron-Jobs, Testing & Go-Live

Erinnerungen, externe Sync, Testflow

7.1 — Cron-Jobs definieren

Datei: services/cron.js

```

const db = require('../db');
const { sendEmail } = require('./email');
const { syncExternalCalendars } = require('./ical-sync');

// JOB 1: 24h-Erinnerung – täglich um 09:00
async function send24hReminders() {
  const bookings = await db.any(`
    SELECT b.*, p.company_name AS provider_name, st.name AS service_name
    FROM bookings b
    JOIN providers p ON p.id = b.provider_id
    JOIN provider_services ps ON ps.id = b.service_id
    JOIN service_templates st ON st.id = ps.service_template_id
    WHERE b.status = 'confirmed'
    AND b.starts_at::date = (NOW() + INTERVAL '1 day')::date
    AND b.id NOT IN (
      SELECT related_id FROM email_log
      WHERE template_id = 'booking_reminder_24h'
    )
  `);
  for (const b of bookings) {
    await sendEmail('booking_reminder_24h', b.customer_email, b);
  }
}

// JOB 2: 1h-Erinnerung – alle 15 Minuten
async function send1hReminders() {
  const bookings = await db.any(`
    SELECT b.*, p.company_name AS provider_name, p.email AS provider_email,
      st.name AS service_name
    FROM bookings b
    JOIN providers p ON p.id = b.provider_id
    JOIN provider_services ps ON ps.id = b.service_id
    JOIN service_templates st ON st.id = ps.service_template_id
    WHERE b.status = 'confirmed'
    AND b.starts_at BETWEEN NOW() + INTERVAL '50 min' AND NOW() + INTERVAL '70 min'
    AND b.id NOT IN (
      SELECT related_id FROM email_log
      WHERE template_id = 'booking_reminder_1h'
    )
  `);
  for (const b of bookings) {
    await sendEmail('booking_reminder_1h', b.customer_email, b);
    await sendEmail('booking_reminder_1h', b.provider_email, b);
  }
}

// JOB 3: Rechnung nach erbrachter Leistung
async function sendInvoicesForCompleted() {
  const bookings = await db.any(`
    SELECT b.*, p.company_name AS provider_name
    FROM bookings b
    JOIN providers p ON p.id = b.provider_id
    WHERE b.status = 'confirmed' AND b.ends_at < NOW()
    AND b.id NOT IN (SELECT related_id FROM email_log WHERE template_id = 'invoice_ready')
  `);
  for (const b of bookings) {
    await db.none('UPDATE bookings SET status = $1 WHERE id = $2', ['completed', b.id]);
    await sendEmail('invoice_ready', b.customer_email, b);
  }
}

// Schedule
setInterval(syncExternalCalendars, 15 * 60 * 1000);
setInterval(send1hReminders, 15 * 60 * 1000);
setInterval(send24hReminders, 60 * 60 * 1000);
setInterval(sendInvoicesForCompleted, 30 * 60 * 1000);

```

7.2 — End-to-End-Test-Szenario

Schritt	Aktion	Erwartetes Ergebnis
1	Anbieter registriert sich auf klard.de	E-Mail welcome_provider
2	Anbieter aktiviert Stripe im Dashboard	Onboarding-Link → Stripe-Flow → Rücksprung
3	Anbieter trägt externe iCal-URL ein	Verbindung getestet, Slots blockiert
4	Kunde sucht & bucht Leistung	Stripe-Checkout öffnet sich
5	Kunde zahlt mit Testkarte 4242...	Webhook → status=confirmed → 2 E-Mails
6	Termin erscheint im Anbieter-Kalender	iCal-Feed liefert das Event nach max. 5 Min.
7	24h vor Termin	Cron sendet booking_reminder_24h
8	Termin endet	status=completed, invoice_ready-E-Mail
9	Stripe-Auszahlung nach 7 Tagen	Anbieter erhält 93% (100% – 7% Provision)

7.3 — Go-Live Checkliste

- Replit Always-On-Plan aktiviert (für Cron-Jobs)
- Custom Domain klard.de auf Replit gerichtet
- Stripe Test-Keys durch Live-Keys ersetzt
- Webhook im Stripe-Dashboard mit Live-URL aktualisiert
- Resend-Domain klard.de verifiziert (DKIM + SPF gesetzt)
- Backup-Strategie für PostgreSQL eingerichtet (Supabase automatisch)
- Monitoring: Sentry oder LogTail für Error-Tracking
- Datenschutzerklärung + AGB für Anbieter und Kunden online
- Cookie-Banner mit Consent-Management (z.B. Cookiebot)
- Erste 5 Pilot-Anbieter über Energy-Impuls-Netzwerk eingebucht

✓ Bereit zum Launch

Wenn alle Punkte erledigt sind, können Sie Klard öffentlich machen. Empfohlene Reihenfolge: erst Soft-Launch mit Energy-Impuls-Netzwerk (10-20 Anbieter), nach 2-3 Wochen Test-Buchungen → öffentliche Marketing-Kampagne mit Pressemitteilung zur dena-Pflicht 2027.